

Liskov Substitution Principle C++



explain liskov substitution principle and give examples in c++

The **Liskov Substitution Principle (LSP)** is one of the five SOLID principles of object-oriented design. It states:

"Objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program."

In other words, if class `S` is a subclass of class `T`, then objects of type `T` should be replaceable with objects of type `S` **without altering the desirable properties of the program** (correctness, task performed, etc).

🧠 Key Idea

A subclass should **extend** the behavior of the parent class, not **restrict** it.

Violating LSP usually happens when:

- Subclasses override methods in a way that breaks expected behavior.
- Subclasses throw exceptions where the base class wouldn't.
- Subclasses invalidate the assumptions made in the base class.

✅ Example that Follows LSP (C++)

```
cpp

#include <iostream>
using namespace std;

class Bird {
public:
    virtual void fly() {
        cout << "This bird can fly." << endl;
    }
};

class Sparrow : public Bird {
public:
    void fly() override {
        cout << "Sparrow is flying." << endl;
    }
};

void makeBirdFly(Bird* bird) {
    bird->fly();
}

int main() {
    Bird* bird = new Sparrow();
    makeBirdFly(bird); // Output: Sparrow is flying.
}
```

```
delete bird;
return 0;
}
```

✓ Explanation:

- `Sparrow` is a subclass of `Bird`.
- `makeBirdFly` expects a `Bird\*` and works fine with a `Sparrow\*`.
- This respects LSP because the substitution doesn't break functionality.

✗ Example that Violates LSP

```
cpp

#include <iostream>
using namespace std;

class Bird {
public:
    virtual void fly() {
        cout << "This bird can fly." << endl;
    }
};

class Penguin : public Bird {
public:
    void fly() override {
        throw logic_error("Penguins can't fly!");
    }
};

void makeBirdFly(Bird* bird) {
    bird->fly();
}

int main() {
    Bird* bird = new Penguin();
    makeBirdFly(bird); // Throws exception
    delete bird;
    return 0;
}
```

✗ Why this breaks LSP:

- `Penguin` inherits from `Bird` but doesn't behave like a `Bird` should.
- Calling `fly()` on a `Bird\*` that's actually a `Penguin\*` causes an exception.
- This violates the expectation that all `Bird`s can fly.

🔧 Fixing the Design

We can fix this by restructuring our class hierarchy:

cpp

```

`class Bird {
public:
    virtual void eat() = 0;
};

class FlyingBird : public Bird {
public:
    virtual void fly() = 0;
};

class Sparrow : public FlyingBird {
public:
    void fly() override {
        cout << "Sparrow flies!" << endl;
    }
    void eat() override {
        cout << "Sparrow eats seeds." << endl;
    }
};

class Penguin : public Bird {
public:
    void eat() override {
        cout << "Penguin eats fish." << endl;
    }
};

```

Now, `Penguin` is not forced to implement `fly()`, and the LSP is respected.

Want me to give an example using custom shapes or bank accounts instead?



what should exactly be substitutable? base class or derived class

Great question — this gets to the **core** of the Liskov Substitution Principle.

🔄 What should be substitutable?

The **derived class (subclass)** should be **substitutable in place of** the **base class**.

In other words:

Wherever a **base class object** (or reference/pointer) is expected, you should be able to **pass in a derived class object** without breaking functionality, correctness, or expectations.

🧠 Think of it like this:

- The **base class** defines the **contract** (methods, behaviors, guarantees).
- The **derived class** must **honor** that contract.
- If the derived class **changes behavior**, removes functionality, or adds unexpected side effects — it's **not substitutable**, and **LSP is violated**.

Example Analogy (Real-world):

Imagine:

- You have a `Vehicle` base class with a `startEngine()` method.
- A `Car` and a `Truck` both inherit from `Vehicle` — great, both have engines.
- But someone makes a new class `Bicycle : public Vehicle`, and overrides `startEngine()` to throw an error —  now you can't safely substitute a `Bicycle` where a `Vehicle` is expected.
That's a violation of LSP.

Summary

What is substitutable?	 Correct Way	 Wrong Way
Base class	Is the expected type	Should not be passed into subclass contexts
Derived class	Must be able to replace base class	Must not break the expected behavior

Want to go deeper with a more complex example (e.g. shape area calculation, file I/O, or user permissions)?